

Roost – Use of Encryption

Export compliance documentation for the iOS app · 3 October 2026

1. Product

App name	Roost
Bundle ID	me.holzeis.roost.roost (with extensions ...ShareExtension and ...NotificationServiceExtension)
Version	1.0.0
Developer	Richard Holzeis
Description	A private family chat app: text messages, photos and videos, video calls and live location sharing. It connects only to a chat server that the family runs on its own hardware, reached over the family's private network. The app is free.
Source code	Publicly available at https://github.com/holzeis/roost

2. Summary

- The app uses only standard, published encryption algorithms (IETF RFCs). It contains **no proprietary or non-standard cryptography**.
- Encryption is used only to protect the user's own communications (message previews in notifications, call media) and to connect to services. It is not user-configurable, and the app offers no general-purpose encryption to users or other apps.
- Some of this encryption is performed by Apple's operating system (CryptoKit, Keychain, URLSession/TLS). Some is performed by libraries bundled in the app (WebRTC's BoringSSL, the Flutter engine's BoringSSL, and the Dart *cryptography* package).
- The app is a mass-market consumer product, distributed only through the App Store, free of charge.
- The app is not made available in France.

3. Encryption functions

Function	Algorithms and key length	Standards	Implementation
Message previews in push notifications: the server encrypts each preview to the receiving device; the notification service extension decrypts it before display	X25519 key agreement (256-bit), HKDF-SHA-256 key derivation, ChaCha20-Poly1305 authenticated encryption (256-bit key)	RFC 7748, RFC 5869, RFC 8439	Apple CryptoKit (operating system)
Storage of the device's private key for the above	iOS Keychain data protection	—	Apple Keychain Services (operating system)
Video and voice calls: encryption of call media between the app and the family's own media server	DTLS 1.2 key exchange (ECDHE), SRTP with AES-128 (AES-CM / AES-GCM)	RFC 6347, RFC 5764, RFC 3711, RFC 7714	WebRTC (via the LiveKit and flutter_webrtc SDKs), bundled BoringSSL

Same preview encryption as the first row, for the app's Android version. The code is compiled into the iOS app but not executed on iOS	X25519, HKDF-SHA-256, ChaCha20-Poly1305 (256-bit)	RFC 7748, RFC 5869, RFC 8439	Dart <i>cryptography</i> package (open source)
HTTPS connections made by bundled SDKs: Firebase Cloud Messaging and Google Maps	TLS 1.2 / 1.3	RFC 5246, RFC 8446	Apple URLSession (operating system)
TLS support in the Flutter engine, available to the app's Dart code for HTTPS	TLS 1.2 / 1.3	RFC 5246, RFC 8446	Flutter engine, bundled BoringSSL

The app's connection to the family's own chat server is HTTP and WebSocket inside the family's private network. That network's encryption (WireGuard, provided by the separate Tailscale app) is not part of Roost.

4. Not used

- No proprietary, custom or non-standard algorithms.
- No encryption that users can configure, select or apply to their own data or to other apps.
- No encryption of files or storage beyond iOS's own data protection.
- No network infrastructure, VPN or cryptographic functionality offered to other software.

5. Classification and distribution

- Self-classified by the developer as mass-market encryption software (ECCN 5D992.c) under the U.S. Export Administration Regulations, using only standard cryptography.
- Distributed only through the App Store, free of charge, to the general public. Not made available in France.

Contact: Richard Holzeis, via the App Store Connect account that submitted this app.